

Utvecklingsmodell - Artifact Formats v1

Status: Normative v1 candidate

Companion specification: Visual Language v1

Source format: YAML

1. Purpose

Artifact Formats v1 defines the semantic source formats for Context, Pulse, and UI. The YAML artifacts are authoritative. Rendering is derived from them using Visual Language v1.

The formats are intentionally small and artifact-specific. v1 does not introduce a generic metamodel, global registry, UUID scheme, presentation fields, or normative cross-artifact references.

2. Common rules

- Parsing is strict. Unknown fields are rejected.
- IDs are machine-readable, stable within their natural artifact scope, and unique where the artifact requires identity.
- Names are human-readable display text and may change without changing identity.
- Presentation and renderer concerns do not belong in semantic YAML.
- Generated D2, SVG, and PNG files are derived artifacts and are not semantic sources.
- References must resolve within the artifact.

3. Context format

Context describes one system, directly communicating people/external systems, domain data flows, data direction, and optional initiative.

```
context:
  system:
    id: ev-charging
    name: EV Charging

  parties:
    - id: user
      type: person
      name: User

    - id: mercedes
      type: external-system
      name: Mercedes

    - id: electricity-price-service
      type: external-system
      name: Electricity Price Service

    - id: garo
      type: external-system
      name: GARO

  flows:
    - id: charging-preferences
      from: user
      to: ev-charging
      name: Charging preferences
      initiative: user

    - id: charging-information
      from: ev-charging
      to: user
      name: Charging information

    - id: vehicle-state
      from: mercedes
      to: ev-charging
```

```
name: Vehicle state
initiative: ev-charging
```

3.1 System

system is required and identifies exactly the system described by the artifact. It contains exactly id and name.

3.2 Parties

parties contains directly communicating external parties. Each party has id, type, and name. Allowed v1 types are person and external-system.

3.3 Flows

Each flow has id, from, to, and name. from -> to always defines data direction.

initiative is optional and references the actual system/party ID that can initiate the interaction giving rise to the flow. It is deliberately not encoded as from/to, because initiative and data direction are independent semantics.

When initiative is omitted, initiative is unspecified. Opposite data directions carrying different domain information are modeled as separate flows.

3.4 Context validation

- Exactly one system is present.
- System ID and all party IDs are mutually unique.
- Every flow ID is unique.
- Every from/to reference resolves to the system or a party.
- from and to must reference different endpoints.
- initiative, when present, resolves to either the flow's from endpoint or its to endpoint.
- Party type is exactly person or external-system.
- Flow names describe domain data, not protocol operations.

4. Pulse format

Pulse describes possible causal event propagation using Behaviors, Pulses, and Flows. Trigger is a role in a Flow, not a separately declared semantic element.

```
pulse:
  behaviors:
    - id: observe-vehicle-soc
      name: Observe vehicle SoC

    - id: plan-charging
      name: Plan charging

  pulses:
    - id: vehicle-observation
      display: "01"
      name: Vehicle observation

    - id: vehicle-state-updated
      display: "02"
      name: Vehicle state updated

  flows:
    - trigger: Startup
      pulse: vehicle-observation
      to: observe-vehicle-soc

    - from: observe-vehicle-soc
      pulse: vehicle-state-updated
      to: plan-charging
```

4.1 Behaviors

Each Behavior contains `id` and `name`. A Behavior is something the system does in reaction to a Pulse.

4.2 Pulses

Each Pulse contains `id`, `display`, and `name`. The `ID` is semantic identity. `display` is the short diagram identity such as `01` and is not used for references.

4.3 Flows

A Flow contains exactly one source form: either free-text `trigger` or Behavior reference `from`. It also contains `pulse` and Behavior reference `to`.

The invariant is therefore **trigger XOR from**. A Flow with both is invalid; a Flow with neither is invalid.

4.4 Pulse validation

- Behavior IDs are unique.
- Pulse IDs are unique.
- Pulse display values are unique within the artifact.
- Every pulse reference resolves to a declared Pulse.
- Every from reference resolves to a declared Behavior.
- Every to reference resolves to a declared Behavior.
- Every Flow contains exactly one of `trigger` and `from`.
- `trigger` is non-empty domain text when present.
- Cycles and fan-out are valid and require no special syntax.
- No validation rule infers Pulse relations from data dependencies.

5. UI format

UI describes user-visible Views, their Information and Actions, and only meaningful Navigation between Views.

```
ui:
  views:
    - id: planning
      name: Planning

    actions:
      - id: select-target-soc
        name: Select target SoC

      - id: select-deadline
        name: Select deadline

      - id: return-to-automatic-control
        name: Return to automatic control

    information:
      - id: current-soc
        name: Current SoC

      - id: desired-target-soc
        name: Desired target SoC

      - id: effective-target-soc
        name: Effective target SoC

      - id: deadline
        name: Deadline

    navigation:
      - to: economics

  - id: economics
    name: Economics
    information:
```

```
- id: charging-cost
  name: Charging cost
```

5.1 Views

Each View contains `id` and `name`. `actions`, `information`, and `navigation` are optional when the View has none of that kind.

5.2 Actions

Each Action contains `id` and `name`. The name is a verb phrase expressing user intent, not a gesture or UI control.

5.3 Information

Each Information item contains `id` and `name`. The name is a noun or noun phrase representing a user-relevant concept independent of presentation.

5.4 Navigation

Navigation has no ID and no name in v1. A navigation entry contains only `to`, referencing another View.

```
navigation:
- to: economics
```

Its complete semantics are: the user can navigate from the containing View to the referenced View. Navigation is included only when that path has user significance; generic global access need not be modeled.

5.5 UI validation

- View IDs are unique.
- Action IDs are unique within their containing View.
- Information IDs are unique within their containing View.
- Every `navigation.to` reference resolves to a declared View.
- A View must not navigate to itself unless such self-navigation later gains explicit semantics; v1 rejects it.
- Navigation entries contain only `to`; `name`, `id`, `style`, `control`, `gesture`, or `layout` fields are invalid.
- Declared ordering of Actions and Information is semantically preserved for deterministic rendering.

6. Deliberately absent from v1

The following are intentionally not part of Artifact Formats v1:

- generic element/relation metamodels
- areas, hierarchy, groups, tags, metadata bags
- layout coordinates or renderer style
- UI widgets, gestures, responsive rules, colors, typography
- protocol details and implementation architecture
- UUIDs, namespaces, or global ID registries
- normative cross-artifact references
- active/superseded lifecycle machinery
- named Navigation relations

7. Source layout and generation

```
system/
  context.yaml
  pulse.yaml
  ui.yaml
```

```
context.yaml -> context.d2 -> context.svg
```

```
pulse.yaml    -> pulse.d2    -> pulse.svg  
ui.yaml       -> ui.d2       -> ui.svg
```

The YAML files are the semantic sources. D2 and SVG are generated. Rendering follows Visual Language v1 and must not add, remove, or reinterpret semantics.

8. v1 design principle

Keep each artifact small, explicit, artifact-specific, strict, and human-readable. Add structure only when an observed semantic need requires it.

Requirements and element links - normative v1 addendum

Date: 2026-09-18. Status: Normative supplement to Artifact Formats v1.

This addendum extends v1 with requirements attached to identified model elements. Existing Context, Pulse and UI schemas remain unchanged. If an earlier statement excludes normative cross-artifact references, the local requirement targets defined here are an explicit, limited exception. No generic cross-artifact relation mechanism is introduced.

A1. Authoritative sources

`context.yaml`, `pulse.yaml` and `ui.yaml` are authoritative semantic model sources. The companion `requirements.yaml` is the authoritative source for requirements and their attachment to elements in those three artifacts. All four files belong to the same system model and are reviewed together.

Requirements must not be embedded in model names, generated D2 or SVG. The requirement badge, click behavior and layout are derived presentation defined by the companion Visual Language addendum.

A2. Requirements format

The document has exactly one top-level field, `requirements`. Its value is a mapping from target address to an ordered, non-empty list of non-empty requirement strings. An empty mapping is valid when no requirements are defined. Duplicate mapping keys are invalid and must be detected before a parser overwrites them.

```
requirements:
  context.flow.operating-data:
    - The system shall initiate retrieval of operating data.
  pulse.behavior.fetch-data:
    - One run shall comprise the complete retrieval.
  ui.action.diagnostics.acknowledge-error:
    - Acknowledgement shall be persistent.
```

Requirement list order is preserved. v1 adds no individual requirement IDs, metadata, status fields, presentation fields or verification fields. A string's wording does not serve as an ID. List positions are not stable identities.

A3. Target addresses

Target addresses use the following forms. Words outside angle brackets are literal. IDs are taken from the corresponding semantic YAML; names and Pulse display numbers are never used as references.

- `context.system.<system-id>` targets the Context system.
- `context.party.<party-id>` targets a Context party.
- `context.flow.<flow-id>` targets a Context data flow.
- `pulse.behavior.<behavior-id>` targets a Behavior.
- `pulse.pulse.<pulse-id>` targets a Pulse.
- `ui.view.<view-id>` targets a View.
- `ui.action.<view-id>.<action-id>` targets an Action in that View.
- `ui.info.<view-id>.<information-id>` targets Information in that View.

Addresses are case-sensitive. For requirement addressing, an ID must not contain a period, which is the address separator. Validation must report an unaddressable ID rather than silently renaming it.

Pulse Flows, free-text triggers and View Navigation have no declared semantic IDs in v1 and are not directly addressable by this format. Relevant requirements must target an identified element, such as the triggering Pulse or affected View. This supplement does not introduce identities for those constructs.

A4. Attachment semantics

Each mapping entry attaches its requirements directly to exactly the addressed element. There is no implicit inheritance from a View to its contents, from a Behavior to its emitted Pulses, or between artifacts. A renderer must not infer or redistribute attachments.

One requirement string may be explicitly repeated at several targets if that is intended. Such repetition does not imply shared requirement identity. An element without an entry has no directly attached requirements; this says nothing about whether other requirements affect its behavior.

A5. Validation

- Reject unknown root fields, non-mapping requirement collections, invalid target syntax, duplicate keys, empty lists and non-string or blank requirement values.
- Resolve every target against the corresponding Context, Pulse or UI artifact of this system model. Reject unknown IDs and wrong element kinds. Action and Information IDs resolve only within their addressed View.
- Apply all existing model validation rules as well as the addressing restriction above. Never repair invalid references by matching display names.
- Reject orphaned targets after deleting or renaming an element. Update the model and its requirements together; do not silently discard requirements.
- Missing entries for otherwise valid elements are allowed. An empty `requirements` mapping is allowed.

A6. Review and generation

The reviewed model comprises the three artifacts and the requirement attachment file. Diagram review must provide access to the exact requirement strings attached to the selected element. Approval of diagrams alone must not be assumed to approve requirements that were omitted from the review.

Generation reads all four sources. Model notation follows Visual Language v1. Requirement indicators are derived solely from resolved, non-empty attachments and follow its Requirements addendum. Generated files must not add, omit, summarize or reinterpret the authoritative requirement strings shown during review.